

1 Literature Review & Related Work

1.1 Key Literature Sources

To establish a foundational background for generating, simulating, and implementing a RISC-V System-on-Chip (SoC) using the Chipyard framework on an FPGA target, key prior works across core hardware generators, simulation environments, and FPGA prototyping flows were reviewed.

1.1.1 Core Generators and Framework Architecture

The foundational architecture of the Chipyard framework is introduced by Amid et al. [?], establishing an integrated agile design environment that unifies core generators, interconnects, and peripheral tools using the Chisel HDL [?]. Central to this ecosystem is the Rocket Chip SoC generator described by Asanović et al. [?], which provides parameterized single/multicore TileLink-based processor subsystems. For evaluations requiring high-performance out-of-order processing, Celio et al. [?] detailed the Berkeley Out-of-Order Machine (BOOM), offering a configurable open-source superscalar core integrated directly within the Chipyard ecosystem.

1.1.2 FPGA Emulation, Simulation, and Toolchains

System-level verification and hardware prototyping draw heavily on software co-simulation and FPGA acceleration techniques. Fast, cycle-exact C++ model creation via Verilator [?] allows for early software validation prior to dynamic hardware synthesis. For FPGA-targeted acceleration, Karandikar et al. [?] introduced FireSim, demonstrating how host-target transformations enable cycle-exact FPGA acceleration for Chipyard-generated RISC-V architectures. Furthermore, resource utilization trade-offs for soft-core processor deployment on FPGAs were contextualized by evaluating open-source microcontroller platforms such as PULPino [?]. Finally, the privilege execution levels required for baremetal execution and Supervisor-mode OS booting are anchored in the RISC-V Privileged ISA Specification [?].

1.2 Comparative Analysis: Chipyard/Chisel Agile Design vs. Traditional Verilog/VHDL Methodologies

1.2.1 The Classical Approach: Verilog and VHDL

Traditional System-on-Chip development relies heavily on Hardware Description Languages (HDLs) such as Verilog, SystemVerilog, and VHDL. Developed in the 1980s, these languages were originally designed for modeling and simulating discrete logic circuits rather than generating complex, highly parameterizable, multi-core system architectures.

While conventional HDLs offer fine-grained control over gate-level implementation and register-transfer level (RTL) timing, they present several structural limitations when applied to modern heterogeneous SoC design:

- **Limited Abstraction & Reusability:** Traditional HDLs lack modern software engineering paradigms such as object-oriented programming, parametric polymorphism, and functional metaprogramming. Parameterization in Verilog (`defparam` / `parameter`) or VHDL (`generic`) is often restricted to basic structural values (e.g.,

bus widths or array depths) and struggles to express complex conditional topologies, such as arbitrary cache hierarchy topologies or dynamic interconnect routing.

- **Manual Integration Overhead:** In classical workflows, interconnecting processor cores, memory buses (e.g., AXI, AHB), memory management units (MMUs), and peripheral blocks requires manual wiring or rigid, GUI-driven IP integration environments (e.g., AMD Vivado IP Integrator). This process is prone to human error, difficult to version-control, and labor-intensive to refactor across different hardware configurations.
- **Verification Bottlenecks:** Verification in traditional flows relies on separate hardware verification languages (e.g., SystemVerilog UVM) or C/C++ co-simulation wrappers, creating a disconnect between the design specifications and testbench architectures.

1.2.2 Agile Hardware Design with Chisel

To address the productivity gap in hardware design, Constructing Hardware in a Scala-Embedded Language (**Chisel**) introduces hardware construction primitives embedded within the high-level Scala programming language. Chisel does not compile directly to gates; instead, it acts as a metaprogramming language that executes Scala code to generate an intermediate representation known as **FIRRTL** (Flexible Intermediate Representation for RTL), which is subsequently lowered into structural Verilog.

Key architectural advantages provided by Chisel include:

1. **Object-Oriented & Functional Metaprogramming:** Designers can leverage abstract classes, traits, and higher-order functions to write hardware design *generators* rather than static *instances*. For example, an arbiter or interconnect crossbar can be procedurally generated based on arbitrary Scala data structures evaluated at compile time.
2. **Type Safety & Object Parametricity:** Chisel enforces strict type checking during hardware elaboration. Signals, interfaces, and bundled records (e.g., Decoupled interfaces implementing handshake protocols) are verified prior to Verilog generation, eliminating structural configuration errors early in the design cycle.
3. **Intermediate Representation (FIRRTL):** By decoupling the high-level Chisel specification from the target Verilog, FIRRTL enables automated compilation passes. Transformations—such as clock-gating insertion, memory mapping, or instrumentation for FPGA emulation—can be applied programmatically to the FIRRTL AST (Abstract Syntax Tree) without modifying the source design.

1.2.3 Integrated Ecosystem: The Chipyard Framework

While Chisel provides the design language, **Chipyard** provides the integrated agile SoC development framework. Chipyard combines a suite of Chisel-based core generators—such as the in-order **Rocket** core and the out-of-order **BOOM** (Berkeley Out-of-Order Machine)—alongside TileLink interconnect generators, memory controllers, and peripheral devices into a unified, version-controlled repository.

Compared to traditional Verilog IP integration workflows, Chipyard provides a fundamentally different paradigm, as detailed in Table ??.

Table 1: Comparison of Traditional Verilog/VHDL Design and Chipyard/Chisel Generator Methodology

Dimension		Traditional Verilog / VHDL Design	Chipyard / Chisel Generator Methodology
Design Abstraction		Register-Transfer Level (RTL) netlists / instances	Metaprogrammed hardware <i>generators</i> in Scala
System Interconnect		Manual wiring of AXI/AHB buses or GUI IP Integrators	Programmatically elaborated TileLink / AXI crossbars
Parameterization		Static macros, localparams, and simple generics	Dynamic Scala-based configuration objects evaluated at elaboration
Target Portability		Manual adaptation per target board or ASIC process	Uniform elaboration targeting C++ simulators (Verilator), FPGAs, or ASIC tools
Verification Cycle		UVM / SystemVerilog testbenches, manual simulation setups	Integrated C++ co-simulation (Verilator), QEMU, and FPGA-accelerated FireSim

1.2.4 Trade-offs and Considerations for FPGA Prototyping

While Chisel and Chipyard dramatically improve design velocity and architecture-level exploration, transitioning from high-level dynamic generators to physical FPGA deployment introduces specific trade-offs:

- **Compilation and Elaboration Latency:** Chisel designs require a multi-stage compilation flow (Scala elaboration → FIRRTL transformations → Verilog generation → Vivado synthesis). The elaboration stage can consume significant memory and CPU time for large, heterogeneous multi-core configurations.
- **Readability of Generated RTL:** The Verilog emitted by the Chisel compiler is machine-generated structural netlist code. While fully functional and synth-ready, debugging post-synthesis timing closure issues or signal routing at the generated Verilog level can be significantly more challenging than in hand-written Verilog.
- **Resource Utilization Optimization:** Hand-crafted VHDL or Verilog can be micro-optimized for specific FPGA primitives (e.g., direct instantiation of Xilinx DSP48 slices or specialized BRAM configurations). Chisel generators abstract these features away, relying on the FPGA synthesis tool (e.g., Vivado) or specialized memory mapping FIRRTL passes to infer optimal target-specific hardware resources correctly.

2 Detailed Review of Key Literature Sources

To support the design, simulation, and implementation methodology of a RISC-V System-on-Chip (SoC) architecture on FPGA, the following key literature sources were evaluated across three foundational pillars: SoC generation frameworks, simulation and FPGA acceleration tools, and privileged software execution architectures.

2.1 Primary SoC Generation & Ecosystem Frameworks

Chipyard: Integrated Agile RISC-V SoC Design Environment

- **Citation:** Amid et al. (2020) [?]
- **Summary:** Introduces the Chipyard framework, an open-source, agile hardware development environment developed at UC Berkeley. Chipyard integrates core generators (e.g., Rocket, BOOM), domain-specific accelerators (e.g., Hwacha, Gemini), interconnect generators (TileLink), peripherals, and verification tools into a single, cohesive ecosystem managed via Scala/Chisel and FIRRTL.
- **Thesis Relevance:** This paper serves as the primary reference for the SoC construction flow. It provides the architectural and toolchain rationale for selecting Chipyard over traditional, manual netlist integration workflows.

The Rocket Chip Generator

- **Citation:** Asanović et al. (2016) [?]
- **Summary:** Describes the Rocket Chip SoC generator written in Chisel. It details the tile-based architecture, the single-issue in-order Rocket core pipeline (RV64IMAFDC), parameterized cache hierarchies (L1 instruction/data caches, L2 inclusive cache), and the TileLink interconnect protocol.
- **Thesis Relevance:** The Rocket core acts as the baseline core configuration evaluated in this thesis for both baremetal execution and Linux boot flows. Citing this work justifies the microarchitectural parameter choices (e.g., pipeline depth, cache sizing, and ISA extensions).

BOOM: The Berkeley Out-of-Order Machine

- **Citation:** Celio et al. (2015) [?]
- **Summary:** Details the architecture of BOOM, an open-source, physical register-file-based superscalar out-of-order RISC-V core generator written in Chisel. It outlines how BOOM implements speculative execution, register renaming, reorder buffers (ROB), and branch prediction.
- **Thesis Relevance:** Essential for the comparative performance and resource utilization analysis. It explains the microarchitectural overheads (increased LUT/FF utilization and critical path constraints) observed when contrasting out-of-order execution against baseline in-order pipelines on FPGA targets.

Chisel: Constructing Hardware in a Scala Embedded Language

- **Citation:** Bachrach et al. (2012) [?]
- **Summary:** Introduces Chisel, a hardware construction language embedded in Scala. It explains how high-level programming paradigms—such as object-oriented design, functional metaprogramming, and parameterized types—are used to construct dynamic hardware generators that compile down to FIRRTL intermediate representations and Verilog netlists.

- **Thesis Relevance:** Provides the foundational hardware language context. This source is used in the literature review to contrast modern object-oriented hardware generation against traditional Verilog/VHDL design methodologies.

2.2 Simulation, Verification, & FPGA Prototyping

Verilator: Fast Open-Source C++ HDL Simulator

- **Citation:** Snyder (2004) [?]
- **Summary:** Details the internal mechanics of Verilator, an open-source tool that converts Verilog and SystemVerilog specifications into cycle-exact, multithreaded C++ executable models for high-speed software simulation.
- **Thesis Relevance:** Directly justifies the pre-synthesis software verification flow. It explains why Verilator is leveraged to verify baremetal binaries and software workloads with 100% cycle-exact fidelity prior to executing long synthesis runs on FPGA hardware.

FireSim: FPGA-Accelerated Cycle-Exact Scale-Out System Simulation

- **Citation:** Karandikar et al. (2018) [?]
- **Summary:** Introduces FireSim, a framework that transforms Chisel/FIRRTL SoC specifications into cycle-exact, FPGA-accelerated simulation models targeting cloud-based or local FPGA hardware.
- **Thesis Relevance:** Serves as a key theoretical reference for FPGA prototyping methodology, demonstrating how host-target transformations operate within the broader Chipyard toolchain during FPGA deployment.

PULPino: An Open-Source Microcontroller System-on-Chip

- **Citation:** Traber et al. (2016) [?]
- **Summary:** Presents PULPino, an open-source single-core microcontroller platform featuring 32-bit RISC-V cores. It provides comparative empirical data regarding maximum clock frequency (f_{max}), power consumption, and FPGA resource utilization (LUTs, FFs, BRAMs).
- **Thesis Relevance:** Provides a comparative reference point in the evaluation section, establishing a baseline to measure the resource footprint and interconnect overheads of 64-bit Chipyard TileLink systems against lightweight microcontroller-class soft cores.

2.3 Software Flow & Privileged Execution

The RISC-V Instruction Set Manual, Volume II: Privileged Architecture

- **Citation:** Waterman and Asanović (2019) [?]

- **Summary:** Defines the hardware privilege levels of the RISC-V ISA: Machine Mode (M-mode for baremetal/bootloaders), Supervisor Mode (S-mode for OS kernels), and User Mode (U-mode for applications). It specifies virtual memory address translation (Sv39/Sv48), control status registers (CSRs), and trap handling.
- **Thesis Relevance:** Directly supports the software deployment scope of the thesis. It supplies the theoretical groundwork for executing M-mode baremetal workloads versus booting S-mode operating systems (Linux) requiring an intermediate Supervisor Binary Interface (OpenSBI).

References

- [1] A. Amid, D. Biancolin, A. Gonzalez, K. Grubb, S. Karandikar, H. Liew, A. Magellan, A. Ou, N. Pemberton, A. Rigge, et al., “Chipyard: Integrated agile RISC-V SoC design environment,” *IEEE Micro*, vol. 40, no. 4, pp. 35–45, 2020.
- [2] J. Bachrach, H. Vo, B. Richards, Y. Lee, A. Waterman, R. Avizienis, J. Wawrzynek, and K. Asanović, “Chisel: Constructing hardware in a Scala embedded language,” in *Proceedings of the 49th Annual Design Automation Conference (DAC)*, 2012, pp. 1216–1225.
- [3] K. Asanović, R. Avizienis, J. Bachrach, S. Beamer, D. Biancolin, C. Celio, H. Cook, D. Ditzel, B. Elies, R. Gimenez, et al., “The Rocket Chip Generator,” University of California, Berkeley, Tech. Rep. UCB/EECS-2016-17, 2016.
- [4] C. Celio, P. F. Chiu, B. Nikolic, D. Patterson, and K. Asanović, “BOOM: The Berkeley Out-of-Order Machine,” University of California, Berkeley, Tech. Rep. UCB/EECS-2015-167, 2015.
- [5] W. Snyder, “Verilator: Fast open-source C++ HDL simulator,” *IEEE Design & Test of Computers*, 2004.
- [6] S. Karandikar, H. Mao, D. Kim, D. Biancolin, A. Amid, A. Lee, N. Pemberton, E. Amarel, P. Rigge, K. Asanović, et al., “FireSim: FPGA-accelerated cycle-exact scale-out system simulation in the public cloud,” in *Proceedings of the 45th Annual International Symposium on Computer Architecture (ISCA)*, 2018, pp. 29–42.
- [7] A. Traber, M. Gautschi, A. Pullini, P. D. Schiavone, D. Rossi, and L. Benini, “PULPino: An open-source microcontroller system-on-chip,” in *International Workshop on Power and Timing Modeling, Optimization and Simulation (PATMOS)*, 2016.
- [8] A. Waterman and K. Asanović, “The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 1.11,” *RISC-V Foundation*, Tech. Rep., 2019.